Self-Attention in the Transformer Encoder

Core idea: Self-attention lets every token look at all other tokens and decide which tokens are relevant for building a context-aware representation.

1. Intuitive Example

Consider: **“The animal was tired.”** When processing the token **“tired”**, the model can attend strongly to **“animal”** because the animal is the entity whose state is being described. The output representation of “tired” therefore contains contextual information from the other words.

Key point: Attention is not a hard lookup. It creates a weighted combination of information from multiple tokens.

2. Where Self-Attention Fits

Input embeddings → **Positional encoding** → **Multi-Head Self-Attention** → **Add & LayerNorm** → **Feed-Forward Network** → **Add & LayerNorm** → **Output**

Encoder self-attention is **unmasked**: every token can attend to every token, including tokens on both sides.

3. Q, K and V

For input matrix X, learned projection matrices create:

$$\mathbf{Q = XW_Q} \quad \mathbf{K = XW_K} \quad \mathbf{V = XW_V}$$

Query: what am I looking for? **Key:** what do I offer for matching? **Value:** what information should I provide?

4. Worked Numerical Example

To make the calculation easy to follow, use a tiny sequence of **3 tokens** with a query/key/value dimension of **2**. We will focus on the attention output for the first token.

Assume the already-projected matrices are:

Token	Q	K	V
Token 1	[1, 0]	[1, 0]	[10, 0]
Token 2	[0, 1]	[1, 1]	[0, 10]
Token 3	[1, 1]	[0, 1]	[5, 5]

Step 1 — Compute QK^T

For Token 1, its Query is Q_1 = [1, 0]. Compare it with every Key:

```
Q1·K1 = [1,0]·[1,0] = 1
Q1·K2 = [1,0]·[1,1] = 1
Q1·K3 = [1,0]·[0,1] = 0
```

Therefore, the first row of QK^T is:

$$\mathbf{[1, 1, 0]}$$

Step 2 — Scale by $\sqrt{d_k}$

Here d_k = 2, so $\sqrt{d_k} = \sqrt{2} \approx 1.414$.

$$\mathbf{[1, 1, 0] / \sqrt{2} \approx [0.707, 0.707, 0]}$$

Step 3 — Apply Softmax

Softmax converts these scores into weights that sum to 1:

$$\mathbf{Softmax([0.707, 0.707, 0]) \approx [0.401, 0.401, 0.198]}$$

Interpretation: for Token 1, Tokens 1 and 2 receive the strongest attention, while Token 3 receives less.

Step 4 — Weighted Sum of Values

Use the attention weights to combine the Value vectors:

$$\text{Output}_1 = 0.401[10,0] + 0.401[0,10] + 0.198[5,5]$$

$$= [4.01, 0] + [0, 4.01] + [0.99, 0.99]$$

$$\text{Output}_1 \approx [5.00, 5.00]$$

So Token 1's new representation is no longer based only on Token 1. It contains information gathered from all three tokens, weighted according to relevance.

5. The Full Matrix View

The above calculation was for one Query. In the real operation, every Query is compared with every Key at once:

$QK^T \rightarrow n \times n$ attention-score matrix → scale → row-wise Softmax → $n \times n$ attention-weight matrix → multiply by V → $n \times d_v$ output matrix.

Example with 3 tokens: the attention matrix is 3 × 3. Row 1 tells how Token 1 distributes attention across Tokens 1–3; Row 2 does the same for Token 2; Row 3 for Token 3.

6. Complete Self-Attention Equation

$$\text{Attention}(Q,K,V) = \text{Softmax}((QK^T) / \sqrt{d_k})\, V$$

7. Why It Is Called Self-Attention

In encoder self-attention, Q, K and V are all derived from the **same input sequence**. This allows tokens within the sequence to interact with one another.

8. Multi-Head Self-Attention

The encoder normally uses multiple attention heads. Each head has separate learned projections and can learn different relationships. The head outputs are concatenated and passed through a final linear projection.

Input → Head 1, Head 2, …, Head h → Concatenate → Linear projection → Multi-head attention output

9. Encoder vs Decoder Self-Attention

Property	Encoder	Decoder
Mask	No causal mask	Causal mask
Future tokens	Visible	Hidden during generation
Q/K/V source	Same encoder sequence	Same decoder sequence

10. What Self-Attention Produces

The output is a **contextualized representation for every token**. The same word can therefore receive different representations in different contexts.

Self-attention mixes information across positions; the Feed-Forward Network then applies a learned nonlinear transformation to each position.

11. Interview-Ready Answer

"In encoder self-attention, every token generates a Query, Key and Value using learned linear projections. We calculate QK^T to measure pairwise relevance, divide by $\sqrt{d_k}$ for stability, apply Softmax to obtain attention weights, and use those weights to compute a weighted sum of the Value vectors. This gives every token a contextualized representation. Encoder attention is unmasked, so each token can attend to the entire input sequence."

12. Key Takeaways

- Q asks what information a token needs.
- K determines how well each token matches that query.
- V contains the information that gets aggregated.
- QK^T gives an $n \times n$ relevance matrix.
- Divide by $\sqrt{d_k}$ before Softmax.

- Softmax gives normalized attention weights.
- Weighted V produces contextualized representations.
- Encoder self-attention is unmasked.
- Multi-head attention performs this process in parallel with different learned projections.